CSA3023/CSE3023/CSM3023
WEB-BASED APPLICATION DEVELOPMENT

Lab Modules

Wan Nural Jawahir Hj Wan Yussof, Rabiei Mamat,
Mohd Arizal Shamsil Mat Rifin
& Muhammad Syaffiq Hazari

Contents

List of Figures

Lab Module 6

JSP: SAVING AND RETRIEVING DATA FROM DATABASE

6.1 Objectives

The primary goal of this lab is to enable students to integrate JavaServer Pages with a relational database management system. Upon completion of this lab, students will be able to:

1. **Establish Database Connectivity**: Successfully connect a JSP application to a MySQL database using the JDBC (Java Database Connectivity) driver.
2. **Perform CRUD Operations**: Implement SQL transactions to Create (insert) and Read (retrieve) records through JSP scriptlets and standard actions.
3. **Implement Data Validation**: Use Regular Expressions within JavaBeans to constrain and validate user input before database insertion.
4. **Apply Design Patterns**: Utilize JSP Model 1 architecture and Data Access Objects (DAO) to manage business logic and separate database interactions from the presentation layer.
5. **Handle Errors**: Create dedicated error pages using JSP directives to manage and display exceptions during database transactions.

6.2 Introduction

In modern web development, the ability to persist and retrieve data is fundamental. This lab exercise focuses on JSP-Database integration, teaching students how to move beyond static data and simple form processing to create data-driven web applications.

Students will learn the lifecycle of a database transaction in a web environment: starting services via a control panel, creating database schemas and tables using phpMyAdmin, and writing Java code to bridge the gap between the user interface and the storage layer. The lab also introduces advanced concepts like JavaBeans for data encapsulation and the DAO pattern for cleaner, more maintainable code structures.

6.3 Software Requirements

To complete the tasks in this manual, the following software environment is required:

- **Web & Database Server Bundle: XAMPP**: to provide the Apache web server and MySQL database.
- **Database Management Tool: phpMyAdmin**: for schema and table creation
- **Integrated Development Environment (IDE)**: NetBeans.
- **Database Driver**: MySQL Java Connector.
- **Web Browser**: Any modern browser (e.g., Google Chrome, Mozilla Firefox, Microsoft Edge) for testing and viewing output.

6.4 Task 1: Using JSP Page to Access a Simple MySQL Database

Objective	Write a JSP that can insert data to MYSQL database as "Welcome to access MySQL database with JSP...!" and also display steps of how to connect with MYSQL database.
Problem Description	• Create a table known as `FirstTable` using database schema CSA3203, create the first column as a character length 45. • Create `SampleInsertionRecord.jsp` page to process and acknowledge the user upon inserting record in the database
Estimated Time	20 minutes

6.4.1 Step 1 - Create a table namely `FirstTable` using phpMyAdmin

1. Start XAMPP control panel.
2. Start the Apache web server.
3. Start the MySQL database
4. Click the *Admin* button for MySQL.
5. Go to *Database*'s tab.
6. Key-in as `CSA3203` and click *Create* button.
7. Database schema successfully created.
8. Use any tool to manipulate the SQL statement. Create table `FirstTable` in the created database schema.
9. Create *FirstTable*'s table.
10. Execute the SQL statement
11. Table successfully created.

6.4.2 Step 2 - Create `SampleInsertionRecord.jsp` to insert data in *FirstTable* table.

1. Go to `C:\CSA3023` and create sub-directory as *Lab 6*
2. Go to NetBeans.
3. Go to File -> New Project
4. Select Java Web -> Web Application
5. Click the *Next* button.
6. Type Project Name: *Lab6*
7. Choose Project Location `C:\CSA3023\Lab6`.
8. Click the *Next* button.
9. Select Server: *Apache Tomcat*.
10. Select Java EE Version: Java EE 7 Web.
11. Click the *Next* button.
12. Click the *Finish* button.
13. Create a new JSP's page for and rename `SampleInsertionRecord`.
14. Type header1 as ***Lab 6 - Task 1 - Sample Insertion records into MySQL through JSP's page***.
15. To support the database driver, we need to use JSP Page Directive to provide directions and instructions to the container.
16. Write the following code:
17. Save and compile `SampleInsertionRecord.jsp` file.
18. Run the `SampleInsertionRecord.jsp` file and sample of output is shown below:

6.4.3 Step 3 – go to the database.

1. Go to Database schema (CSA3023)
2. Click on-> CSA3023 -> FirstTable -> then Browser (see the data is already there!!)

Reflection

1. How do you want to submit specific information from one form to next form?
2. What happened if the field name you specify in `request.getParameter("`*field_name*`")` in second page is different from the field name you defined in first page?

6.5 Task 2: Create Records via JSP Page

Objective	Using JSP to insert records retrieve from MySQL database.
Problem Description	1. Create a table known as **Author** using database schema CSA3023 using these attributes: • **authno** as a character length 15 and must be primary key name. • **address** as a character length 40. • **city** as a character length 40. • **state** as a character length 40. • **zip** as a character length 40. 2. Create `insertAuthor.jsp` as a main interface to register a new author. 3. Create `processAuthor.jsp` page to process and acknowledge the user upon inserting record in the database.
Estimated Time	40 minutes

1. Use any tool to manipulate the SQL statement. Create table ***author*** in CSA3023 database schema.
2. Create ***author***'s table.
3. Execute the SQL statement.
4. Table successfully created.
5. Create a new JSP page and rename as insertAuthor.

6. Write an HTML code to
   (a) Display six (6) labels and textfields representing *Author No*, *Name*, *Address*, *City*, *State* and *Zip* (in the combo box).
   (b) Create a *Submit* button and *Cancel* button.
   (c) Upon submission, redirect the page to processAuthor.jsp page.
7. Produce the following output:
8. Go to *Lab6* project folder.
9. Right click -> *New* -> *Java Class*.
10. Click *Java Class*.
11. Rename *Class Name* as *Author* and *Package* as *lab6.com*.
12. Define **six (6)** instance variables for *Author* class.
13. Define the *getter* and *setter* method for corresponding attributes.
14. Create a new JSP page as `processAuthor`.
15. Add the page directive to `processAuthor.jsp page`.
16. Create an *author*'s object using JSP Standard Action tag.
17. Assign data entry from page insertAuthor.jsp page into *author*'s bean.
18. Load the database driver and create a connection to the database.
19. Create a *PreparedStatement*'s object.
20. Execute the query and display the result.
21. Close database connection.
22. Save and compile `prosessAuthor.jsp` file.

IMPORTANT: Please add **MySQL Java connector** to your project before running the program.

23. Run `insertAuthor.jsp` page.
24. Key-in the record.
25. Click *Submit* button.
26. The record will save in the database, and user get a notification.

Reflection

1. What have you learnt from this exercise?
2. Define step by step before you successfully perform the transaction in a database.

6.6 Task 3: Create Records Constrained by Regular Expression in JSP

Objective	Using JSP Standard Action, scriptlets and regular expression to insert records retrieve from MySQL database.
Problem Description	1. Create a table known as **Student** using database schema CSA3023 using these attributes: • **stuid** as a character length 15 and must be primary key name. • **stuname** as a character length 50. • **stuprogram** as a character length 40. • **address** as a character length 40. 2. Create `insertStudent.jsp` as a main interface to register new book. 3. Create `processStudent.jsp` page to process and acknowledge the user upon inserting record in the database. 4. Create `displayStudent.jsp` page to populate records. 5. Create `errorStudent.jsp` page to handle an error.
Estimated Time	50 minutes

6.6.1 Step 1 – Create a table `student` using phpMyAdmin.

1. Create table ***student*** in CSA3023 database schema.
2. Execute the SQL statement.
3. Table successfully created.

6.6.2 Step 2 – Create `insertStudent.jsp` as a main interface to register a new student.

1. Create a new JSP page and rename as insertStudent.
2. Write an HTML code to
   (a) Display three (3) labels and textfields representing *Author No*, *Name*, and *Program* (in the combo box).
   (b) The first field must be started with captain letters the numbers input
   (Use the following regular expression in *Book JavaBeans* file)
   (c) Create a *Submit* button and *Cancel* button.
3. Produce the following output:

6.6.3 Step 3 – Create `Book JavaBeans`

1. Go to *Lab6* project folder.
2. Right click -> *New* -> *Java Class*.
3. Click *Java Class*.
4. Rename *Class Name* as *Book* and *Package* as *lab6.com*.
5. Define **three (3)** instance variables for *Book* class.
6. Define the *getter* and *setter* method for corresponding attributes.
7. Define *getter* and *setter* method plus regular expression for *stuno* attribute.

6.6.4 Step 4 – Create `processBook.jsp` to insert record into the database

1. Create a new JSP page as `processStudent`.
2. Add the page directive to `processAuthor.jsp page`.
3. Create a *student*'s object using JSP Standard Action tag.
4. Assign data entry from page `insertStudent.jsp` page into *Student*'s bean.
5. Load the database driver and create a connection to the database.
6. Create a *PreparedStatement*'s object.

7. Execute the query and display the result.

8. Close database connection.

6.6.5 Step 5 – Create `errorBook.jsp` to display any error message

1. Create a new JSP page as `errorStudent`.

2. Define the page directive to declare that this is an error page.

3. Complete remaining of code.

4. Save and run `errorStudent.jsp` file.

6.6.6 Step 6 – Running the program and create a new database

1. Run `insertStudent.jsp` page.

2. Key-in the record.

3. Click `Submit` button.

4. The record will be saved in the database, and user will get a notification.

Reflection

1. What have you learnt from this exercise?

2. Define step by step before you successfully perform the transaction in a database.

6.7 Task 4: Perform Retrieving Records Via JSP Page

Objective	Use Java Scriptlet to query a list of records.
Problem Description	Retrieve student records and populate in the table.
Estimated Time	30 minutes

1. Run program ***insertStudent.jsp*** from Task 3.

2. Insert the following records;

UK12489	Ahmad Salam	BSc with IM
UK56789	Rosnah Azman	BSc Soft. Eng.
UK67342	Liew Cheng Huat	BSc in Robotics

3. Go to *Lab6* project.
4. Create a new JSP file.
5. Key-in filename as `queryStudent`.
6. Rename title as `Lab 6 - Task 4`.
7. Rename `h1` as `Lab 6 - Task 4: Retrieving record via JSP Page`.
8. Use JSP page directive to include the information such as content type, and use Java SQL API.
9. Use a Java scriptlet to create a simple structure of HTML table.
10. Then, load the database driver and connect to the database.
11. Create Statement for the query.
12. Perform query to retrieve records from the Student's table.
13. Fetch the record into HTML table.
14. Close the database connection.
15. Enhance the CSS for the table.
16. Save *queryStudent.jsp*, then run.
17. You should get the following output.

Reflection

1. What have you learnt from this exercise?
2. Explain the differences when using *Statement()* and *PreparedStatement()*.

6.8 Task 5: Create A Record Using JSP Model 1

Objective	Use JavaBeans to perform SQL transaction.
Problem Description	Create a sample web form to register the Marathon event.
Estimated Time	40 minutes

1. Choose Project *Lab6*.
2. Create a new JSP file.
3. Key-in filename as `registerMarathon`.
4. Prepare the following Graphical User Interface (GUI).
5. Create a JavaBeans `Marathon`.
6. Create a `Database` class that has two methods; `getConnection()`, and `closeConnection()`.
7. Create a `MarathonDAO` class to perform SQL transaction for business object `Marathon` and store it into `package lab6.com`.
8. Create a new file name known as `processMarathon.jsp`.
9. Import related classes in `package lab6.com`.
10. Instantiate an object `Marathon`.
11. Create a Java Scriptlet to invoke respective object for inserting record in `marathon`'s table.
12. Save and run `processMarathon.jsp`, and key-in related record.
13. The output will appear in a web browser.

Reflection

1. What have you learnt from this exercise?
2. Describe the benefits of using JavaBeans.

Exercise

Implement user login

1. Create a table known as **userprofile** using database schema CSA3023 using these attributes

   - **username** as a character length 15 and must be primary key
   - **password** as a character length 10
   - **firstname** as varchar(50)
   - **lastname** as varchar(50)

2. Create **insertUser.html** as the main interface to register a new user.

3. Create **processUser.jsp** page to process the record.

4. Create **login.jsp** page to login to the system.

5. Create **doLogin.jsp** to validate username and password. If validation succeeds, redirect the page to **main.jsp** page that displays the username, firstname, and lastname

6. However if fails, redirect the page to **login.jsp** with message 'Invalid username or password..!'